

JTS PowerTool

Technical Details

Project flow, setup, connections, file structure and technology stack

Updated to the current production setup · 5 October 2026

Replaces the earlier version of this document (adds Keys & Connections, GitHub App, Jira, website builder,
Slack key capture, billing and invoices)

1. What the System Does

JTS PowerTool is a Slack-powered AI assistant with an admin dashboard. Each client company gets its own channels, its own connections (GitHub, Jira, API keys) and its own usage and billing.

Capability	What it means
Chat assistant	Anyone in a client channel mentions the bot (or sends a direct message) and Claude answers using the conversation memory, live web search and the client's connected tools.
GitHub work NEW	Clients connect their <i>own</i> GitHub with one click. The bot reads repositories, and proposes issues, file changes and pull requests that need approval.
Jira work NEW	Clients connect their own Jira Cloud site with one click. The bot searches and reads issues directly; creating, updating, commenting and moving issues need approval.
Website builder NEW	The bot asks for the stack, builds a complete site as a draft, and after one approval creates the repository in the client's GitHub (and publishes it on GitHub Pages where possible). Later edits go through pull requests.
Client API keys NEW	Each client manages its own keys in <i>Keys & Connections</i> or by posting <code>KEY_NAME = value</code> in Slack. The client's Anthropic key is used for its replies so JTS does not bill them.
Approvals (human in the loop)	Every action that changes something outside the chat (GitHub, Jira, websites) posts an approval card. It is approved in Slack or on the dashboard.
Usage & billing	Token use and USD cost are logged per message and channel. JTS can generate invoices per client and mark them paid.

Who uses what

Role	Can do
JTS Admin	Everything: all clients, users, global API keys, system settings, activity log, conversation inspector, database view.
Client Admin	Their own client only: dashboard, approvals, usage & billing, clients & channels, and managing the client's keys and GitHub / Jira connections.
Team Member	Their own client only: <i>My Activity</i> , approvals (view / approve as allowed) and a read-only view of Keys & Connections.

A client user is always limited to the channels of their own client. A client user who is not yet assigned to a client sees nothing.

2. How the Project Works

Step by step, in simple language.

1 Webhook trigger

- A user mentions the bot, or sends a message, in a Slack channel (or a direct message).
- Slack immediately sends an HTTP POST to `/api/slack/events`. Button clicks go to `/api/slack/interactive`. (A Microsoft Teams webhook also exists in `app/teams`.)

2 Security and signature check

- The backend computes an HMAC-SHA256 with `SLACK_SIGNING_SECRET` and compares it with `x-Slack-Signature`.
- Requests older than 5 minutes are rejected (replay protection). A bad signature returns `401`.

3 Deduplication

- Slack retries an event when it does not get an answer within about 3 seconds.
- The backend records each message id (`record_processed_activity`) and drops repeats, so the bot never answers twice.

4 Key messages are handled first NEW

- If the whole message is `KEY_NAME = value` lines, the system saves the key, deletes the Slack message and replies by name only. The message never reaches the database, the logs or Claude. See section 6.
- All other messages continue below.

5 Fast answer and queue hand-off

- The backend returns `200 OK` within milliseconds and stores a job in the PostgreSQL `job_queue` table.
- The separate `jts-worker` service polls the queue (about once a second), claims a job and does the slow AI work. (The earlier version of this document mentioned Redis; the queue is now PostgreSQL.)

6 Context, history and memory

- Recent conversation turns for the thread or channel (default rolling window of 6) come from PostgreSQL.
- Semantic memory uses local embeddings (`BAAI/bge-small-en-v1.5` , 384 dimensions) to bring back relevant past knowledge.
- The user's name and timezone are added so times are shown locally.
- Anything that looks like a key is replaced with `[hidden]` before it is saved and when old messages are loaded. NEW

7 Client and key selection NEW

- The channel is matched to its client folder (`channel_metadata`). Slack's real channel id and a stored legacy alias are treated as the same channel.
- Which Anthropic key answers: the channel's own key, then the client's key, then the JTS key. Only usage on the JTS key is billed to the client.
- Website building uses a stronger model (default `claude-sonnet-5-5`) with a cost cap; normal chat uses `claude-haiku-4-5`.

8 Permission check and Claude's tools NEW

Claude decides which tool to use. The set of tools depends on what the client has connected:

Tool group	Tools	Runs
Web	<code>search_web</code> , <code>fetch_url_content</code>	Directly
GitHub (client's own account)	read issues, files, code, commits, pull requests; write issues, files, branches, pull requests	Reads directly, writes after approval
Connect buttons	<code>connect_github</code> , <code>connect_jira</code>	Posts a one-time button
Jira (when connected)	<code>jira_search_issues</code> , <code>jira_get_issue</code> , <code>jira_list_projects</code> ; <code>jira_create_issue</code> , <code>jira_update_issue</code> , <code>jira_add_comment</code> , <code>jira_transition_issue</code>	Reads directly, changes after approval
Website builder	<code>ask / set stack</code> , <code>draft files</code> , <code>publish_website</code> , <code>start_site_edit</code> , <code>propose_site_changes</code> , <code>list_my_websites</code>	Drafts in the database, publish after approval
Keys	<code>list_saved_keys</code> (names only), <code>add_api_key</code> (opens the secure form)	Directly

- Clear requests (for example "create a Jira task" or "list the keys") force a real tool call, so the bot never only promises to do something.
- The bot is told to prefer Jira tools for Jira project keys such as `KAN-12` and GitHub tools only for repositories.

9 Human approval

- Every change posts an approval card in the Slack conversation with **Approve & Apply** and **Reject**. The same request appears on the dashboard *Approvals* page.
- A request is valid for 24 hours and can be claimed once, so two people cannot approve it twice.
- After approval the action runs with the **client's own** GitHub or Jira connection, never with a shared JTS credential (unless the JTS fallback for GitHub is still allowed).

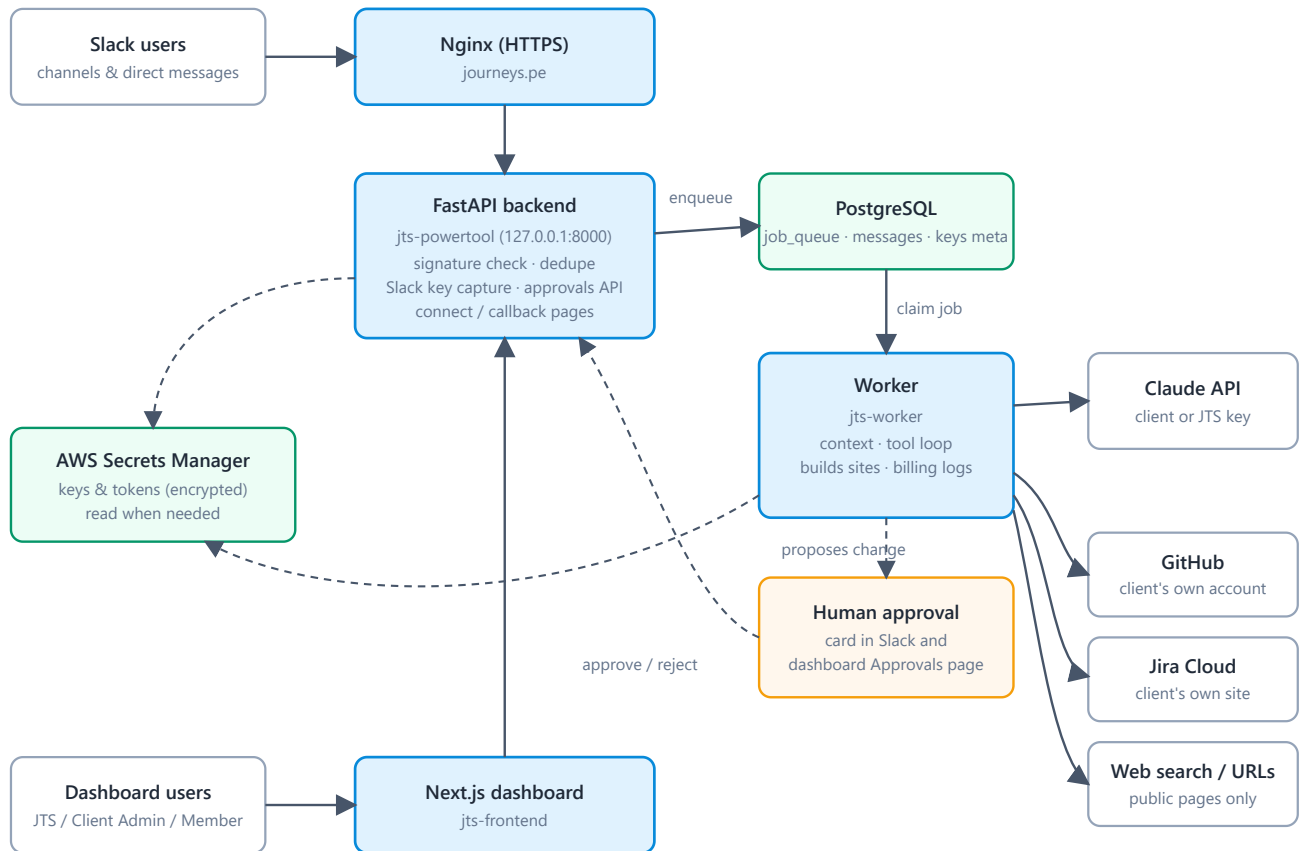
10 Reply and cost logging

- The final answer is posted to the Slack thread with `chat.postMessage` .
- Telemetry stores input and output tokens and the USD cost per message; this feeds *Usage & Billing*. Replies made with a client's own Anthropic key are stored with zero billable cost.
- Security-relevant actions (key saved or removed, connections changed) are written to the activity log. Key values are never logged.

Who is asking is checked first. Before a GitHub or Jira tool runs, the bot matches the person's Slack email to a dashboard user and checks the nine permission checkboxes on the Users page (section 5). A refused request stops there: no approval card is posted and the bot tells the person why. The approval card is the second safety layer for anything that changes data.

3. System Architecture Diagram

Request, queue, worker, approval and secret flow.



Solid arrows: main request path. Dashed arrows: approvals and secret reads. Slack buttons also reach the backend through Nginx.

4. Slack Setup

Connect Slack to the backend.

01 Install the app in a workspace

- Open `https://journeys.pe/api/slack/install` and choose the workspace from the top-right dropdown in Slack. Click **Allow**.
- The backend receives the workspace id, team name and bot token and stores them in `slack_workspaces` and the secrets vault.
- In the channel, invite the bot (for example `@jpt` / "JTS Powertool Agent"). It then answers when mentioned.

02 Slack app settings (api.slack.com/apps)

Setting	Value
Event Subscriptions URL	<code>https://journeys.pe/api/slack/events</code>
Interactivity URL	<code>https://journeys.pe/api/slack/interactive</code>
OAuth redirect URL	<code>https://journeys.pe/api/slack/oauth/callback</code>
Bot token scopes	<code>app_mentions:read, channels:history, channels:read, chat:write, files:read, files:write, groups:history, groups:read, im:history, im:read, mpim:history, mpim:read, users:read</code> (the app also has <code>users:read.email</code>)
User token scope NEW	<code>chat:write</code> under User Token Scopes , used only so the bot can delete a message that contains a key

03 Let the bot delete key messages **NEW**

- A bot can only delete its own messages. To delete the person's message, JTS uses the user token of a workspace **Owner or Admin**.
- In the Slack app: **OAuth & Permissions** → **User Token Scopes** → **Add** `chat:write`, then **Reinstall to Workspace** signed in as an Owner/Admin. Copy the **User OAuth Token** (`xoxp-`).
- On the dashboard **API Keys** page save it as `SLACK_USER_TOKEN`, then restart `jts-powertool` and `jts-worker`.
- Make sure Slack's *Message editing & deletion* setting lets Owners/Admins delete other people's messages.
- That single token belongs to Axcel World. **Every other workspace needs its own:** on **Clients & Channels** → **Slack workspaces**, the *Delete key messages* column shows On or Off. For an Off workspace, an Owner or Admin of that workspace opens the **Turn on** link while signed in to it and clicks Allow. The token is saved as `SLACK_USER_TOKEN_<workspace id>` and used for that workspace only; a normal member's token is refused.
- If it is off for a workspace, keys are still saved and the bot asks the person to delete their own message.

Re-running `/api/slack/install` installs the bot scopes only; adding `?delete_messages=true` also asks the signed-in admin for the delete permission. The user token is created from the app's own OAuth page. Each Slack workspace (Axcel World, JTS Team) is installed separately.

5. Setting Up a Client

Every client follows the same short path. Do this once per client.

#	Step	Where / how
1	Create the client folder	Clients & Channels → New client folder . A client folder is one company. Use the pencil on the folder to set Billed as (organization) ; invoices for the client are then made out to that organization automatically.
2	Move its Slack channels into the folder	Clients & Channels , "Channels not in a folder yet": pick the client for each channel. Only JTS Admin can move channels. A channel that is not in a client cannot use GitHub, Jira or client keys.
3	Create the client's users	Users & Organizations → Add user . Choose the role (Client Admin or Team Member) and the client folder.
4	The Client Admin connects the tools	Keys & Connections , or in Slack: <code>@bot connect my github</code> , <code>@bot connect my jira</code> . No tokens are typed.
5	The Client Admin adds API keys	Keys & Connections , or in Slack with <code>KEY_NAME = value</code> . Name the Anthropic key <code>ANTHROPIC_API_KEY</code> to use the client's own Claude account.

Each client has its own connections. Connecting GitHub or Jira for one client does not connect it for another. Test from a channel that belongs to the client that was connected.

Several Slack workspaces and personal chats NEW

- **Add a workspace:** sign in to it in the browser and open `https://journeys.pe/api/slack/install`, then click Allow. The server saves the workspace and its bot token automatically. Invite the bot to a channel with `/invite @jpt`.
- **Find Slack channels** (Clients & Channels) searches every connected workspace and records which workspace each channel belongs to. Slack's own built-in Slackbot chat is skipped. Channels and personal chats are both listed under "Channels not in a folder yet" so any of them can be assigned to a client.
- **Link a workspace to a client:** in **Clients & Channels** → **Slack workspaces**, a JTS Admin picks the client for a workspace that belongs to one company. Leave a shared workspace on "Shared".
- For a linked workspace, any chat that is *not* in another client's folder, including a person's **personal chat with the bot**, belongs to that client: its usage goes on that client's invoice, the client's own Anthropic key is used, GitHub / Jira / keys work in it, and the client's admins can see its approvals and usage. A channel placed in a different client's folder always stays with that client.
- In a workspace that is **not** linked, personal chats belong to no client. They use the JTS key and are not invoiced to anyone (the usage is still recorded for JTS Admin).

Tool permissions (GitHub and Jira checkboxes) NEW

On the **Users** → **Permissions** tab, a JTS Admin sets what each person may ask the assistant to do in Slack. There is one row per person with a checkbox for each permission, saved per row (the arrow button puts a row back to its role defaults). The Add and Edit user forms do not contain permissions:

Group	Checkbox	Allows
GitHub	Can read repositories	Read issues, files, code, commits and pull requests
GitHub	Can create and comment on issues	Create issues, update issues and add comments
GitHub	Can commit code and files	Create or update files, push files and create branches
GitHub	Can open pull requests	Open pull requests
GitHub	Can publish and edit websites	Publish a built website and propose changes to a published site
Jira	Can read tickets	Search tickets and read their details
Jira	Can create tickets	Create new tickets
Jira	Can edit and comment	Change title, description, priority, labels; add comments
Jira	Can move or close tickets	Change a ticket's status

- **Older settings carry over:** before the GitHub boxes were split, one box "Can make changes" covered issues, files, pull requests and websites. A person who had it ticked now has all four of the new change boxes ticked, and a person who had it unticked has none, so nobody gains or loses access.
- **Defaults:** JTS Admin and Client Admin start with everything ticked; a Team Member starts with the two "read" boxes only. A new user starts with their role's defaults; change them later on the Permissions tab.
- **JTS Admins** always have every permission and cannot be restricted. Client users can only act in channels of their own client.
- **How the person is found:** the bot reads the person's Slack email and matches it (ignoring upper/lower case) to the email on the Users page. The two emails must be the same.
- **Someone who can't be matched** (their Slack email is not on the Users page) can read but not make changes, and is told to ask an administrator to add them. Two optional secrets change this: `TOOL_PERMISSIONS_UNMATCHED=allow` lets them change things; `TOOL_PERMISSIONS_ENFORCE=false` switches the checks off.
- If the database check itself fails, changes are refused (reads still work) rather than silently allowed. Refusals are written to the activity log as `TOOL_PERMISSION_DENIED`.
- These boxes decide who may *ask*. They do not change who may *approve*; approving is still done by an approver in Slack or on the dashboard.

Approvals visible to a Client Admin

A client user sees only approvals from the channels of their own client. If the Approvals page is empty for a client admin, check in **Users** that the user's client is the one that owns the channel. The server log line `[APPROVALS_SCOPE]` explains exactly what was hidden and why.

6. Connections and Keys

A. GitHub (one-click, client's own account) NEW

- **One-time JTS setup:** register one GitHub App (guide: `docs/setup/github-app-setup.md`) and save `GITHUB_APP_ID`, `GITHUB_APP_SLUG`, `GITHUB_APP_PRIVATE_KEY`, `GITHUB_APP_CLIENT_ID`, `GITHUB_APP_CLIENT_SECRET`, `GITHUB_APP_WEBHOOK_SECRET` on the **API Keys** page.
- **Per client:** anyone in the client channel types `@bot connect my github` (or the Client Admin clicks **Connect GitHub**). A single-use 15-minute link opens GitHub; they choose the account and repositories, then Install and Authorize.
- JTS stores only the installation id in `github_connections`. For every action it mints a 1-hour installation token. The installing user's refresh token is kept in AWS (hidden provider `github_user`) for actions that need a user token, such as creating repositories on personal accounts.
- The Client Admin can pick a default repository and disconnect. `GITHUB_REQUIRE_CLIENT_CONNECTION=true` stops the bot falling back to JTS's own GitHub token.

B. Jira (one-click, client's own site) NEW

- **One-time JTS setup:** create one Atlassian OAuth 2.0 (3LO) app (guide: `docs/setup/jira-app-setup.md`). Scopes `read:jira-work`, `write:jira-work`, `read:jira-user`; callback URL `https://journeys.pe/api/jira/callback`; Distribution set to **Sharing** so other companies can connect. Save `JIRA_OAUTH_CLIENT_ID` and `JIRA_OAUTH_CLIENT_SECRET`.
- **Per client:** `@bot connect my jira` or **Connect Jira** on the card. The user signs in to Atlassian and clicks Accept. The Atlassian account must have a Jira Cloud site.
- JTS stores the site in `jira_connections` and the rotating refresh token in AWS (hidden provider `jira_oauth`). Access tokens last 1 hour. A connection unused for 90 days expires and the card shows "Needs reconnecting".
- The Client Admin can choose a default project for new issues. If a person offers several Jira sites the first is used until they reconnect.

C. Client API keys (Keys & Connections) NEW

- Form: **Key Name** (free text) + **Key Value** + **Applies to** (whole client or one channel). Saving the same name replaces the key.
- The value goes straight to AWS Secrets Manager and is never shown again; the table shows the name, scope, last 4 characters, who saved it and when.
- Only the key named `ANTHROPIC_API_KEY` is used by the assistant today (it is checked with Anthropic before saving). Other keys are stored securely for the client.
- Client Admin edits; Team Members see the list only; JTS Admin can open any client.

D. Saving keys from Slack NEW

Post a message that contains only key lines, for example:

```
OPENAI_API_KEY = sk-proj-abc123xyz987
please save my key SHOPIFY_TOKEN: shpat_abcdef0123456789
MY_TOKEN = abc12345xyz789 for this channel only
```

- The system saves each key for the channel's client, deletes your message, and replies by name only. Values are never stored in the database, logs or conversation memory and are never sent to Claude.
- **What counts as a key:** every line is `NAME = value` or `NAME: value`; the name has no spaces and contains *key, token, secret, password* or *credential* as its own word (or the value looks like a provider key such as `sk-`, `ghp_`, `xoxb-`, `AKIA`); the value is one piece of 8+ characters. Allowed extras: an `@bot` mention, a lead-in like "please save my key", and "for this channel only".
- Anything else is normal chat and is not touched. Direct messages and channels without a client do not save keys (the message is still deleted).
- A value starting `sk-ant-` under a name that contains "anthropic" is stored as `ANTHROPIC_API_KEY`.
- `@bot list the keys` shows names, scope and last 4 characters. Values cannot be retrieved by anyone.
- Every save is written to the activity log as `SLACK_KEY_SAVED` with the person's name. This path trusts everyone in the client's channel.

E. JTS global keys (API Keys page)

- JTS Admin adds global secrets such as `ANTHROPIC_API_KEY`, `SLACK_BOT_TOKEN`, `SLACK_SIGNING_SECRET`, `SLACK_USER_TOKEN`, GitHub App and Jira app secrets. The key name is upper-cased automatically.
- Each is stored in AWS Secrets Manager as `jts-powertool/vault/<name>`; the database keeps only the name and who added it.
- Lookup order when the code needs a secret: the shared bundle (`AWS_SECRET_NAME`), then the individual vault secret, then an environment variable.

7. Website Builder NEW

Ask in Slack: "build a website for a dental clinic". The bot always asks for the stack first; there is no default.

1. **Stack question.** The bot lists the options and waits for the answer in a later message.
2. **Draft.** It writes all files into a draft stored in the database (`site_drafts` , `site_draft_files`). Nothing touches GitHub yet. A private preview link is available for static sites.
3. **Publish with one approval.** The approval card shows the repository name, visibility, files and stack. After approval the repository is created in the client's own GitHub and the code is committed.
4. **Edits.** Later changes are made on a branch, opened as a pull request, and merged only after approval. A conflict check runs first.

Stack	Kind	Result
Static HTML/CSS/JS	static	Published on GitHub Pages (live)
React, Vue, Svelte, Astro, Next.js	build on GitHub	A GitHub Actions workflow is added and Pages is enabled; GitHub builds and publishes
PHP, Laravel, WordPress	code only	Code is committed to the repository; it needs PHP hosting to go live

- Builds use `claude-sonnet-5-5` , a cost cap per build and a time limit. Known sites per client are listed in `client_websites` and on the client page.
- Static sites and the WordPress theme path have been tested live. The framework stacks (React and others) are built but not yet live-tested.

8. Usage, Billing and Invoices

- Each message stores tokens and USD cost in `conversation_messages` and `api_usage_logs` , with the person, channel and model.
- If a client uses its own Anthropic key, its replies are stored at zero billable cost.
- An invoice covers the billable (JTS key) usage of the channels in the client's folder, plus the chats of any Slack workspace linked to that client that are not in another client's folder.
- **Usage & Billing** shows usage per client and channel for a chosen period (week, month or custom range), with CSV export.
- Invoices are generated per client and period (`client_invoices`); a JTS Admin marks an invoice paid manually.
- When an invoice is made, **Bill to (organization)** is filled in from the client automatically: first the organization linked to the client (Clients & Channels, edit the client, *Billed as*), otherwise the organization of a Client Admin assigned to the client. Linking a Client Admin to an organization also links their client if it had none. It can still be changed by hand, and switching client picks up the new client's organization.

9. File Structure

A FastAPI backend with a separate worker, and a Next.js dashboard.

```
jts-powertool/
|-- app/                                # Backend (FastAPI + AI engine)
|   |-- main.py                        # API entry point, login middleware, router wiring
|   |-- slack_router.py                # Slack events, OAuth install, interactive buttons, system prompt
|   |-- worker.py                      # Background worker: claims jobs from job_queue, runs Claude
|   |-- claude.py                      # Claude streaming + tool loop, forced-tool rules, cost cap
|   |-- auth_router.py                 # Login, sessions, roles, client scope
|   |-- api_approvals.py               # Approvals API (list, approve, reject, previews)
|   |-- client_keys_router.py          # Keys & Connections API
|   |-- github_router.py               # GitHub connect/callback/webhook + dashboard endpoints
|   |-- jira_router.py                 # Jira connect/callback + dashboard endpoints
|   |-- site_router.py                 # Website drafts, previews, client websites
|   |-- usage_router.py, invoice_router.py # Usage, billing, invoices
|   |-- channel_secrets_router.py      # Clients & channels, vault (API Keys) endpoints
|   |-- global_settings_router.py, organizations_router.py, logs_router.py, db_router.py, context_router.py
|   |-- memory_manager.py, vector_memory.py # Chat history, semantic memory, key hiding
|   |-- teams/                         # Microsoft Teams webhook
|   |-- services/
|       |-- channel_secrets_service.py # Client/channel keys, folders, AWS Secrets Manager
|       |-- github_app_service.py      # GitHub App connection and tokens
|       |-- jira_service.py            # Jira OAuth connection, tools, token rotation
|       |-- site_builder_service.py    # Website drafts, publish, edit-by-pull-request
|       |-- slack_key_capture.py       # "NAME = value" keys from Slack
|       |-- secret_redaction.py        # Hides key values in text
|       |-- tool_permissions.py        # GitHub / Jira permission checkboxes and the bot's check
|       |-- usage_service.py, invoice_service.py, email_service.py
|   |-- tools/
|       |-- tool_adapter.py            # Tool dispatcher and safety gate (GitHub, Jira, site, keys)
|       |-- slack_approval_ui.py       # Approval cards
|       |-- secrets_manager.py         # Secret lookup (bundle, vault, env)
|       |-- mcp_client.py, web_tools.py
|   |-- db/                           # PostgreSQL session and repositories
|-- frontend/                          # Next.js 14 dashboard
|   |-- src/
|       |-- lib/api.ts, types.ts, permissions.ts # Backend calls, types, date formatters, permission list
|       |-- components/                # Sidebar, Header, DashboardShell, DataTable, ui,
|           |--                               # GithubConnectionCard, JiraConnectionCard,
|           |--                               # ClientApiKeyCard, WebsitesCard, InvoicesPanel,
|           |--                               # UserPermissionsTab, WorkspaceClientsCard
|       |-- app/                       # Pages (see section 10)
|-- docs/plans, docs/setup              # Plans and setup guides (GitHub App, Jira, Slack key messages)
|-- .github/workflows/deploy.yml        # Deploy to EC2 on push to main
`-- requirements.txt, .env.example
```

10. Dashboard Pages

Menu item	Path	Who	What it does
Dashboard	/	JTS, Client Admin	System overview and statistics
My Activity	/my-tasks	Team Member	Their own requests
Approvals	/approvals	All roles	Review, approve or reject GitHub, Jira and website changes; previews and diffs
Usage & Billing	/billing	JTS, Client Admin	Usage by client/channel, periods, CSV, invoices
Clients & Channels	/folders	JTS, Client Admin	Client folders (name, description and the organization a client is billed as), channels, the Slack workspaces table (link a workspace to a client), GitHub / Jira cards, client websites
Keys & Connections	/client-keys	All (edit: Client Admin, JTS)	GitHub and Jira connections, client and channel keys
Users & Organizations	/users , /organizations	JTS	Users tab: accounts, roles, client assignment. Permissions tab: the GitHub / Jira checkboxes for each person
API Keys	/keys	JTS	Global secrets stored in AWS
System Settings	/global-settings	JTS	Timezone and default tool rules
Activity Log	/logs	JTS	Live telemetry and security events
Conversation Inspector	/context	JTS	Stored conversations and prompts
Database	/database	JTS	Read-only database view
Profile Settings	/settings	All	Personal timezone ("System Time")

11. Database Tables

Table	What it stores
dashboard_users , organizations , password_reset_tokens , user_channel_permissions	Accounts, roles, client assignment, the GitHub / Jira permission checkboxes (tool_permissions), set-password links and per-user channel permissions
channel_folders , channel_metadata	Client folders (with the organization each is billed as, organization_id) and the Slack channels assigned to each

<code>slack_workspaces</code>	Installed Slack workspaces, their bot tokens and the client each is linked to (<code>folder_id</code>)
<code>job_queue</code>	Slack messages waiting for the worker (durable queue)
<code>conversation_messages</code> , <code>user_memories</code>	Chat history with per-message tokens and cost; semantic memory (key values hidden)
<code>secrets_vault</code>	Names and owners of the global keys on the API Keys page (values are in AWS)
<code>channel_secret_mappings</code>	Channel-specific keys: provider and the AWS secret name only
<code>folder_api_keys</code> NEW	Client-wide keys: provider, last 4 characters, who saved it, health; also hidden connection tokens (<code>github_user</code> , <code>jira_oauth</code>)
<code>github_connections</code> , <code>github_connect_links</code> NEW	Each client's GitHub installation and the single-use connect links
<code>jira_connections</code> , <code>jira_connect_links</code> NEW	Each client's Jira site and the single-use connect links
<code>pending_approvals</code>	Actions waiting for approval, and the final record of who approved and the result
<code>site_drafts</code> , <code>site_draft_files</code> , <code>site_draft_base_files</code> , <code>client_websites</code> NEW	Website drafts, their files, the base for edits, and each client's published sites
<code>api_usage_logs</code> , <code>client_invoices</code>	Token and cost analytics; generated invoices
<code>telemetry_logs</code> , <code>global_system_settings</code> , <code>claude_context_snapshots</code>	Activity and security events, system settings, saved prompt snapshots

12. Technology Stack

Layer	Technology	Purpose
Frontend	Next.js 14 (App Router), React 18, TypeScript	Admin and client dashboards
Styling and icons	Tailwind CSS, Lucide React	Responsive UI, forms and icons
Backend	FastAPI (Python 3.11+), Uvicorn	REST API, Slack webhook, OAuth callbacks
AI	Anthropic Claude: <code>claude-haiku-4-5</code> (chat), <code>claude-sonnet-5-5</code> (website builds)	Reasoning, code generation, tool selection
Tool protocol	Model Context Protocol (GitHub server) plus direct GitHub App, Jira and web calls	Tool connections
Database	PostgreSQL	Users, channels, queue, chat, billing, connections
Semantic memory	sentence-transformers (BAAI/bge-small-en-v1.5)	Long-term retrieval of past context
Queue and worker	PostgreSQL <code>job_queue</code> + asyncio worker (<code>jts-worker</code>)	Answer Slack in under 3 seconds, work in the background
Secrets	AWS Secrets Manager (boto3)	Every key and token, encrypted
Chat platform	Slack API (Events, Interactivity, OAuth v2); Microsoft Teams webhook	Mentions, threads, cards, buttons
Integrations	GitHub App (REST + Git Data API), Atlassian OAuth 2.0 + Jira Cloud REST API	Repositories, pull requests, issues, tickets
Proxy and process	Nginx, systemd	HTTPS, routing, service restarts on Ubuntu (AWS EC2)
Deployment	GitHub Actions to EC2 over SSH	Push to <code>main</code> deploys and restarts the three services

13. Operations and Known Gaps

Deploy and services

- Pushing to `main` on GitHub runs `.github/workflows/deploy.yml`, which updates the server and restarts `jts-worker`, `jts-powertool` and `jts-frontend`.
- Logs: `sudo journalctl -u jts-powertool` (API) and `-u jts-worker` (worker). Log times are UTC.
- If a deploy seems to have no effect, check for a second service holding port 8000 (`sudo ss -ltnp | grep :8000`). An old leftover `jts-backend.service` once served stale code and has been disabled.

- Useful log tags: `[JIRA]` , `[SLACK_KEYS]` , `[APPROVALS_SCOPE]` , `[GITHUB_APP]` .

Known gaps

- The GitHub and Jira permission checkboxes depend on the Slack email matching the Users page. Someone whose emails differ is treated as "not matched" (read-only) until the email is corrected.
- The Global Settings default checkboxes (stored in the browser) are not used by this check; the per-user boxes and the role defaults are.
- Framework website stacks (React and others) are built but not yet tested live.
- Any member of a client channel can save keys by Slack message (an owner decision); each save is logged by name.
- The Jira search call uses Atlassian's newer search endpoint; if Atlassian changes it, search is the first thing to re-check.
- Old test messages stored before the key-hiding was added may still show in the Conversation Inspector (the bot itself no longer repeats them).

This document describes the production system as of 5 October 2026. Setup guides: `docs/setup/github-app-setup.md` , `docs/setup/jira-app-setup.md` , `docs/setup/slack-key-messages.md` .